ArcBridge

Your AI agent's **architectural brain**.

Plan → Build → Sync → Review

The Problem

AI agents are **brilliant coders** but **terrible architects**.

They start every session nearly blank:

- ❌ No idea **why** code is structured a certain way
- ❌ No awareness of **quality requirements** (security, perf, a11y)
- ❌ No concept of **project phases** or progress
- ❌ No memory of **architectural decisions** already made

> Your agent can write code. It just doesn't know **what the system needs**.

The Workarounds Are Broken

Approach	Why It Fails
`CLAUDE.md` / `.cursorrules`	Static files that go **stale in days**
Spec-driven dev	Specs become **shelfware** by sprint two
Context engineering	Solves **input** — nobody solves the **output feedback loop**
Manual docs	**Negative short-term ROI** — nobody maintains them

The root cause: documentation maintenance has no immediate payoff.

ArcBridge makes it automatic and agent-driven.

What Is ArcBridge?

An **MCP server** + **CLI** that bridges three worlds:

🏗️ Architecture

arc42 documentation
Building blocks, ADRs,
quality scenarios

📋 Planning

Phased workflows
Tasks, gates,
progress tracking

🔍 Code Intelligence

TypeScript Compiler API,
Roslyn (.NET),
Symbol-level indexing

All queryable through **26 MCP tools** your agent already speaks.

One Call. Full Context.

Agent gets a task: "Fix the auth middleware to check token expiry"

```
arcbridge_get_guidance({ file_path: "src/lib/auth/middleware.ts" })
```

Returns instantly:

```
Building Block:    auth-module
Responsibility:    User authentication, session management, authorization
Quality Scenarios: SEC-01 (Auth on all API routes)
                   SEC-02 (No secrets in client bundles)
Related ADRs:      ADR-003 (JWT with refresh tokens)
Existing patterns: verifyToken(), refreshSession(), authGuard()
```

No scanning 20 files. No guessing. **One call.**

The Development Lifecycle

📐 PLAN

Agent loads arc42, quality gates, phase tasks

⚙️ BUILD

Agent codes within arch boundaries & constraints

🔄 SYNC

Agent detects drift, updates docs & task status

👤 REVIEW

Human approves or corrects proposed changes

PLAN → BUILD → SYNC → REVIEW → repeat.

Every cycle makes the project knowledge **more accurate**, not less.

Phase 1: Plan

Before any code, the agent **understands the system**:

- 📐 Loads arc42 building blocks & their responsibilities
- 🎯 Reads quality scenarios (SEC-01, PERF-03, A11Y-02…)
- 📝 Reviews ADRs — decisions already made
- 🗺️ Knows the current phase & pending tasks

> The developer defines intent.
> The agent inherits context.

No more blind starts. No more reinventing decisions.

Phase 2: Build

The agent codes **within architectural boundaries**:

- Knows **which building block** it's working in
- Follows **existing patterns** from actual code — not generic templates
- Respects **quality constraints** (auth on all routes, bundle budgets)
- **Flags boundary crossings** instead of silently adding dependencies

```
arcbridge_get_symbol({ symbol_id: "src/lib/auth/middleware.ts::verifyToken#function" })
```

```
Signature:   (token: string) => Promise<DecodedToken | null>
Dependencies: jwt.decode, db.findSession
Dependents:   authGuard, refreshSession, 3 API routes
```

Phase 3: Sync

After each session, **drift detection runs automatically**:

```
arcbridge_check_drift
```

Catches what humans miss:

- 🆕 New files not mapped to any building block
- 🔗 Undeclared cross-block dependencies
- 🕸️ Building blocks referencing deleted code paths
- 🧪 Quality scenarios with no linked tests

The agent **proposes doc updates** — not the developer writing prose.

Phase 4: Review

The human **reviews a diff, not a blank page**.

```
+ Added src/lib/cache/ to data-access building block
+ Created ADR-007: Redis for session caching
+ Updated dependency graph: auth-module → data-access
- Removed stale code path: src/legacy/auth.ts
```

Accept the drift? → Docs update automatically.

Reject it? → Fix the code instead.

Phase gates enforce quality before moving forward.

Agent Roles — 8 Specialized Agents

Role	Purpose
🏗️ **Architect**	Design, building blocks, ADRs
⚙️ **Implementer**	Feature dev within arch boundaries
🔒 **Security Reviewer**	Auth, secrets, client/server boundary
✅ **Quality Guardian**	Quality scenarios, coverage, perf budgets
📋 **Phase Manager**	Task tracking, drift sync, phase gates
🎨 **UX Reviewer**	Usability, consistency, accessibility
🎓 **Onboarding**	Explain the project using live architecture
👀 **Code Reviewer**	On-demand review: bugs, patterns, simplicity

Each role gets **different context** and **different constraints**.

Before vs. After

	Without ArcBridge	With ArcBridge
Find the right file	Grep + read 10-20 files	1 MCP call
Understand relationships	Manually trace imports	Dependency graph in DB
Know quality constraints	Read docs (if they exist)	Linked to building blocks
Know architectural intent	Ask someone or guess	Building block + ADRs
Context window cost	Thousands of tokens exploring	Targeted, structured responses
Docs over time	Drift silently from code	Drift detection catches it

The Bidirectional Sync

It's not one-way. **Both directions stay in sync.**

Code changes → update docs

1. Drift detection flags mismatches
2. Agent proposes concrete arc42 updates
3. Phase gates block until docs are current

Doc changes → update code context

1. MCP tools call `refreshFromDocs()` automatically
2. Manual YAML edits are visible immediately
3. Task status and progress are preserved

Every coding session makes docs more accurate, not less.

What Gets Generated

```
.arcbridge/
├── config.yaml                    # Project config
├── index.db                       # SQLite — the agent's brain
├── arc42/
│   ├── 01-introduction.md         # Goals & stakeholders
│   ├── 03-context.md              # System boundary
│   ├── 05-building-blocks.md      # Module decomposition → code
│   ├── 06-runtime-views.md        # Key workflows
│   ├── 09-decisions/              # ADRs linked to code
│   └── 10-quality-scenarios.yaml  # Testable requirements
├── plan/
│   ├── phases.yaml                # Phase plan + gates
│   └── tasks/                     # Per-phase breakdowns
└── agents/                        # 8 role definitions
```

Plus platform configs: `CLAUDE.md`, `.github/copilot-instructions.md`

It Teaches You Too

ArcBridge surfaces the **right questions at the right time**:

At init:

> "What's your auth strategy? Here are common patterns with trade-offs."

During implementation:

> "This route has no auth middleware. SEC-01 requires it. Scaffold it?"

At phase boundaries:

> "2 quality scenarios have no tests. 1 building block has undocumented deps."

The goal: make the **informed choice the easy choice**.

Quick Start

```
# Install
npm install -g arcbridge

# Initialize in any project (auto-detects framework)
cd your-project && arcbridge init

# Connect MCP server
# .mcp.json:
{ "mcpServers": { "arcbridge": {
    "command": "npx",
    "args": ["@arcbridge/mcp-server"]
}}}
```

Works with **TypeScript**, **React**, **Next.js**, and **.NET/C#**.

Supports **Claude Code**, **GitHub Copilot**, and any MCP-compatible agent.

ArcBridge

Stop giving your agent **amnesia**.

Start giving it **architectural awareness**.

Plan → Build → Sync → Review → Repeat

It's like giving the agent a senior developer's mental model of the project on day one.